Anexo_Actividad

Contextualización

ConstruYA es una cadena de ferreterías que, desde hace ocho años, opera un sistema de gestión de inventario desarrollado internamente. El sistema cumple su función, pero ha acumulado una considerable deuda técnica: módulos con código duplicado, métodos excesivamente extensos, ausencia de pruebas automatizadas y un historial de cambios disperso, sin un control de versiones ordenado. La gerencia de tecnología ha detectado que cada nueva solicitud de cambio (por ejemplo, la integración con una nueva pasarela de pagos o la corrección de errores reportados por los usuarios) toma cada vez más tiempo y representa un riesgo creciente de introducir nuevas fallas.

Planteamiento del caso

Usted ha sido vinculado(a) como ingeniero (a) de mantenimiento de *software* y su primera tarea consiste en intervenir un componente del sistema (un fragmento de código, un módulo o un repositorio de código abierto de complejidad y propósito similares, previamente autorizado por el docente) aplicando un proceso completo de mantenimiento: diagnóstico, refactorización, gestión de versiones, automatización de pruebas y validación, siguiendo las técnicas y herramientas vistas.

Procedimiento

1. **Selección del sistema o fragmento de código**: seleccione un módulo de código (mínimo 150 líneas) con evidencias de deuda técnica; puede ser un proyecto propio, uno suministrado por el docente o un repositorio de código abierto como: GitHub - emilybache/GildedRose-Refactoring-Kata: Starting code for the GildedRose Refactoring Kata in many programming languages. · GitHub
2. **Diagnóstico**: identifique y documente al menos tres *bad smells* (código duplicado, métodos largos, aglomeración de datos, generalidad especulativa, entre otros) apoyándose en al menos una herramienta de análisis estático (por ejemplo, SonarQube, ESLint, Pylint o similar).
3. **Refactorización:** aplique como mínimo tres principios o técnicas de refactorización estudiadas (por ejemplo, extracción de método, eliminación de código duplicado, renombramiento, simplificación de algoritmos), documentando el código antes y después de cada intervención.
4. **Control de versiones**: gestione todo el proceso en un repositorio Git, utilizando al menos una rama independiente para el trabajo de refactorización, con commits atómicos y mensajes descriptivos. Al finalizar,

integre los cambios a la rama principal mediante una operación de combinación (merge).

5. **Automatización de pruebas y validación:** diseñe y ejecute un conjunto de pruebas automatizadas (unitarias y/o de regresión) con una herramienta de la familia XUnit (JUnit, NUnit, PHPUnit u otra pertinente al lenguaje utilizado) que permita demostrar que la refactorización no alteró el comportamiento externo del sistema.

Resultados esperados y entregables

Elabore un informe técnico (máximo 8 páginas) que documente cada uno de los pasos del procedimiento. El diagnóstico, las decisiones de refactorización, la evidencia de uso de herramientas de análisis y control de versiones, los resultados de las pruebas y una reflexión final sobre el impacto del mantenimiento en la calidad y la sostenibilidad del *software*.

El informe debe entregarse en formato **PDF** y debe evidenciar los resultados de las pruebas automatizadas junto con el enlace al repositorio Git (público o con acceso de lectura habilitado para el docente).

Enlaces de consulta

Bache, E. (s. f.). *GildedRose-Refactoring-Kata* [Software de computadora]. GitHub. https://github.com/emilybache/GildedRose-Refactoring-Kata